

PyBaMM

Training day 2: the model zoo, custom models and performance

PyBaMM Conference 2026 · Ann Arbor, Michigan · 16 October 2026

Today's session: two tracks

30 min

Talk covering both tracks

60 min

Self-guided: pick a track

Model track

01 The model zoo: find, load and test zoo models

02 Writing your own model: a thermal DFN, ready for the zoo

Performance track

03 Input parameters: build once, solve many times

04 Solvers: tolerances, outputs, parallel solves and sensitivities

Model track: the PyBaMM model zoo

Notebook 01

New models need a home



Merge into core?

Core grows with every paper, and the PyBaMM team becomes the gatekeeper of which models people can use. We don't want that role.



Keep it separate?

Hard for others to find, and nothing tests it against new PyBaMM releases, so it can quietly stop working.



The model zoo

Your model lives next to PyBaMM, is tested against every change, is documented automatically, and cites you.

What the zoo gives you

1

manifest file (model.toml) to write

10

standard checks run against your model automatically



Tested

Every change to PyBaMM runs your model, and it is checked against past releases too.



Documented

A docs page and a status badge are generated from your manifest and README.



Cited

Using your model registers your citation, so your work is credited.

Using a zoo model

```
import pybamm
import pybamm_model_zoo as zoo

zoo.list_models()
zoo.info("LinearisedSPM")

model = zoo.load("LinearisedSPM")()
solution = pybamm.Simulation(model).solve([0, 300])
pybamm.print_citations()
```

- LinearisedSPM: an SPM linearised about its starting state, for GITT analysis
- info shows who maintains a model and which PyBaMM versions it supports
- Using a model credits its authors

Anatomy of a zoo model

```
my_model/
├── model.toml
├── __init__.py
├── model.py
├── README.md
├── CITATION.bib
├── tests/
│   └── test_my_model.py
└── examples/
    └── run_my_model.py
```

- model.toml: what the model is, who maintains it, which PyBaMM versions it supports
- model.py: the equations
- README.md: summary, usage, validation and citation
- CITATION.bib: the papers to cite
- tests/: checks that the physics is right
- examples/: scripts showing how to use it

The manifest

```
[model]
slug = "thermal_dfn"
name = "ThermalDFN"
title = "DFN with a lumped thermal energy balance"
class = "pybamm_model_zoo.thermal_dfn:ThermalDFN"
tier = "community"
pybamm_requires = ">=26.10"

[[model.maintainers]]
name = "Your Name"
github = "yourhandle"

[model.citation]
key = "You2026"

[model.tests]
solve_time = 3600
key_variables = ["Voltage [V]", "Temperature [K]"]
```

- pybamm_requires: the PyBaMM versions the model works with
- tier: community or core
- maintainers: who reviews changes to the model
- tests: how long to simulate and which outputs to check

Tested automatically

Valid manifest › Required files › Imports › Cites its authors › Well posed › Builds › Solves

Standard checks

Run on every model: valid manifest, required files present, imports, registers its citation, well posed, builds and solves.

Your physics tests

Check a known result. ThermalDFN matches PyBaMM's DFN when the temperature is held fixed, and conserves energy with no cooling.

Two tiers



community

The default for new models. Tested against every change to PyBaMM; a failure shows on the model's status badge but never blocks a PyBaMM release.



core

Adopted by the PyBaMM maintainers. A failure blocks changes to PyBaMM until it is fixed.

Your model is tested from day one, without ever being able to block a PyBaMM release.

Model track: writing your own model

Notebook 02 · a DFN with a lumped thermal model

A standalone model is a list of equations

```
class ThermalDFN(pybamm.lithium_ion.BaseModel):
    def __init__(self, name="Thermal DFN"):
        super().__init__(name=name)
        pybamm_model_zoo.register_citation(SLUG)

        T = pybamm.Variable("Temperature [K]")
        C = pybamm.Parameter("Cell heat capacity [J.K-1]")
        ...  # DFN: particles, electrodes, electrolyte

        heat_source = current * (ocv - voltage)
        heat_loss = h * area * (T - T_amb)
        self.rhs[T] = (heat_source - heat_loss) / C
        self.initial_conditions[T] = param.T_init
```

- Variables: the unknowns, e.g. temperature
- Parameters: filled in from the parameter set
- Equations, with boundary and initial conditions
- Outputs and voltage cut-offs
- New parameters need default values

Work on it anywhere, then contribute

```
zoo.refresh(external_paths=[Path("custom_model")])
zoo.load("ThermalDFN")
```

Create from the template › **Copy in your files** › **Update the import path** › **Generate the docs** › **Run the checks** › **Open a pull request**

Step-by-step commands: training/day_2/custom_model/README.md

Performance track

Notebooks 03 and 04 · input parameters and solvers

Input parameters: build once, solve many times

```
parameter_values["Current function [A]"] = "[input]"
sim = pybamm.Simulation(model, parameter_values=parameter_values)
for current in currents:
    inputs = {"Current function [A]": current}
    sim.solve([0, 3600], inputs=inputs)
```

~12×

faster per solve for an SPM current sweep

Works for

Currents, diffusivities, reaction rates, temperatures: anything that doesn't change the geometry.

Doesn't work for

Geometric parameters such as electrode thicknesses, which set the mesh.

Solver settings

Output times

Choose output times without forcing the solver to stop at each one.

Tolerances

Trade accuracy for speed. Check with a convergence study.

Output variables

Compute only the outputs you need: less memory, less time.

Parallel solves

Solve a list of inputs together across several threads.

Plus sensitivities: the derivative of any output with respect to input parameters, for parameter fitting.

Over to you

Open training/day_2 and pick a track

Self-guided session

Model track

01 and 02: use zoo models, then add reversible heat to ThermalDFN and re-run the checks.

Performance track

03 and 04: input parameter sweeps, tolerances, parallel solves and sensitivities.

Bring your model

Copy thermal_dfn/, rename it, and get the checks passing. Then open a pull request to the zoo.

github.com/pybamm-team/PyBaMM/tree/main/packages/pybamm-model-zoo